

[CS230] Convolutional Neural Networks: Inception Network & 1x1 Convolutions

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-8. Deep Learning Strategy & Architecture - *Completed*

Chapter 9. Convolutional Neural Networks (Current Part)

- 9.1-9.4 CNN Basics, Classic Nets, ResNet - *Completed*
- **9.5 Inception Network**
 - * The Magic of 1×1 Convolution
 - * Inception Module (Naive vs Optimized)
 - * Bottleneck Layer (Computational Cost Reduction)
 - * Implementation with Keras
- 9.6 Object Detection Introduction - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 ResNet을 통해 신경망을 "깊게(Deep)" 쌓는 법을 배웠습니다. 그런데 구글 연구팀은 전혀 다른 질문을 던졌습니다. "필터 크기를 꼭 하나만 골라야 하나? $1 \times 1, 3 \times 3, 5 \times 5$ 를 다 쓰면 안 되나?" 이 단순하고 무식해 보이는 아이디어에서 출발하여, 연산 효율성을 극대화한 아키텍처가 바로 **Inception Network**입니다. 그리고 이를 가능하게 만든 숨은 공신은 1×1 Convolution입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 CNN의 효율성 혁명을 이끈 Inception 구조와 핵심 기술을 다룹니다.

- **1x1 Conv:** 채널 수를 조절하여 연산량을 줄이는 **Network in Network** 개념을 이해합니다.
- **Inception Module:** 다양한 크기의 필터를 병렬로 수행하고 합치는 구조를 파악합니다.
- **Bottleneck:** 1×1 합성곱을 통해 연산 비용을 **1/10** 수준으로 줄이는 원리를 증명합니다.
- **구현:** 복잡한 분기(Branch) 구조를 Keras로 구현합니다.

2 Essential Terminology

용어	의미	핵심 역할
1×1 Convolution	$1 \times 1 \times C$ 필터 연산	채널 수 조절 (Dimensionality Reduction).
Bottleneck Layer	입력을 압축하는 층	연산량을 획기적으로 줄임.
Inception Module	병렬 연산 블록	다양한 스케일의 특징을 동시에 추출함.
Concatenate	이어 붙이기	병렬로 나온 결과들을 채널 축으로 합침.

3 Core Concepts: 수도꼭지를 잠가라

3.1 1. The Magic of 1×1 Convolution

”교수님, 1×1 필터면 그냥 숫자 하나 곱하는 거 아닙니까?” 2D 이미지에서는 그렇습니다. 하지만 입체적인 볼륨($H \times W \times C$)에서는 다릅니다.

- 연산: $1 \times 1 \times 192$ (입력 채널 수) 크기의 필터가 입력을 훑습니다.
- 효과: 채널 방향으로 FC Layer를 적용하는 것과 같습니다. 필터 개수를 32개로 설정하면, 출력 채널은 32개가 됩니다. ($192 \rightarrow 32$ 압축)

3.2 2. Bottleneck Layer (비용 절감의 핵심)

이 섹션이 오늘 강의의 하이라이트입니다. 숫자로 증명합니다.

□ Computational Cost Analysis (수학적 증명)

상황: 입력 $28 \times 28 \times 192 \rightarrow$ 출력 $28 \times 28 \times 32$ (using 5×5 conv).

1. Naive Approach (그냥 5×5 사용):

$$\text{Cost} = (28 \times 28 \times 32) \times (5 \times 5 \times 192) \approx 120,000,000 \text{ (1.2억)}$$

2. Bottleneck Approach (1×1 로 줄이고 5×5 사용): (1) 1×1 로 192ch $\rightarrow$ 16ch 압축 (중간 단계)

$$(28 \times 28 \times 16) \times (1 \times 1 \times 192) \approx 2,400,000$$

(2) 5×5 로 16ch $\rightarrow$ 32ch 확장 (최종 단계)

$$(28 \times 28 \times 32) \times (5 \times 5 \times 16) \approx 10,000,000$$

총합: $2.4M + 10M = 12.4M$

결과: 연산량이 약 $1/10$ 로 줄었습니다. 성능 저하는 거의 없습니다.

4 Deep Dive: Inception Module Architecture

구글은 고민했습니다. ” 3×3 을 쓸까, 5×5 를 쓸까?” 결론은 ”그냥 다 하자(Do them all)” 였습니다.

- 구조: 1×1 , 3×3 , 5×5 , Max Pooling을 병렬로 수행합니다.
- 병합: 나온 결과들의 크기(28×28)를 맞추고(Padding='same'), 채널 축으로 이어 붙입니다(Concatenate).

- **최적화:** 3×3 과 5×5 앞에는 반드시 1×1 병목 층을 두어 연산량을 줄입니다.

5 Implementation: Inception Block with Keras

Keras Functional API를 사용해야 복잡한 분기(Branch) 구조를 짤 수 있습니다.

```

1 from tensorflow.keras import layers
2
3 def inception_module(x, filters):
4     """
5     x: 입력 텐서
6     filters: 각 분기별 필터 개수 딕셔너리
7     """
8
9     # Branch 1: 1x1 Conv
10    path1 = layers.Conv2D(filters['f1x1'], (1, 1), padding='same', activation='relu')(x)
11
12    # Branch 2: 1x1 (Reduce) -> 3x3
13    path2 = layers.Conv2D(filters['f3x3_reduce'], (1, 1), padding='same', activation='relu')(x)
14    path2 = layers.Conv2D(filters['f3x3'], (3, 3), padding='same', activation='relu')(path2)
15
16    # Branch 3: 1x1 (Reduce) -> 5x5
17    path3 = layers.Conv2D(filters['f5x5_reduce'], (1, 1), padding='same', activation='relu')(x)
18    path3 = layers.Conv2D(filters['f5x5'], (5, 5), padding='same', activation='relu')(path3)
19
20    # Branch 4: MaxPool -> 1x1
21    path4 = layers.MaxPooling2D((3, 3), strides=(1, 1), padding='same')(x)
22    path4 = layers.Conv2D(filters['pool_proj'], (1, 1), padding='same', activation='relu')(path4)
23
24    # 병합 (Concatenate)
25    output = layers.concatenate([path1, path2, path3, path4], axis=3)
26
27    return output

```

Listing 1: Inception Module Implementation

6 FAQ & Pitfalls

Q. 정보를 압축했다가 늘려도 손실이 없나요?

A. 네, 괜찮습니다. 이미지 데이터에는 중복성(Redundancy)이 많기 때문입니다. 1×1 층은 필요한 핵심 정보만 압축(Linear Combination)해서 다음 층에 넘겨주는 역할을 합니다.

Q. Max Pooling 뒤에 왜 1×1 Conv를 붙이나요?

A. 풀링은 채널 수를 줄이지 못합니다. 인셉션 모듈을 거칠 때마다 채널이 계속 늘어나는 것을 막기 위해, 풀링 뒤에 1×1 을 붙여 채널 수를 강제로 줄여줍니다(Projection).

다음 단계 (Next Step)

우리는 이제 이미지를 분류(Classification)하는 최고의 아키텍처들을 모두 섭렵했습니다 (VGG, ResNet, Inception).

하지만 현실 세계에서는 이미지에 무엇이 있는지만 아는 것으로는 부족합니다. ”그 물체가 어디에 있는지(위치)”도 알아야 합니다. 다음 시간에는 컴퓨터 비전의 꽃, [Object Detection]

으로 넘어갑니다. 그중에서도 실시간 객체 탐지의 혁명, **YOLO (You Only Look Once)** 알고리즘을 향한 여정을 시작해 보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **1×1 Conv:** 채널 수를 줄여 연산량을 아끼는 핵심 도구.
2. **Inception:** 여러 필터를 병렬로 사용하여 다양한 특징을 동시에 잡는다.
3. **Bottleneck:** 큰 필터 앞에 1×1 을 두어 입력을 압축한다. (비용 1/10 절감)
4. **Concatenate:** 병렬 연산 결과를 채널 축으로 합친다.